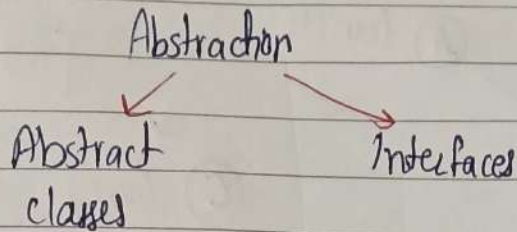


# Interfaces



\* interfaces → defines what an object can do without telling how it does that.

```
interface Car {  
    void drive();  
}
```

```
class Car {  
    void drive() {  
        // do something  
    }  
}
```

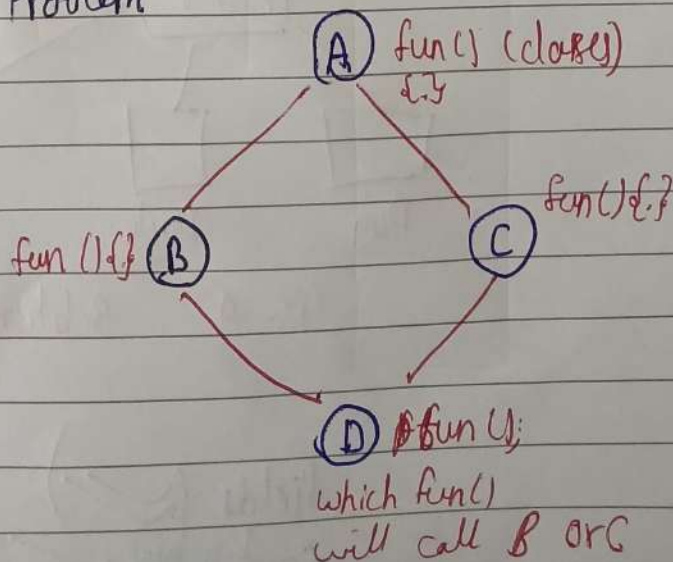
```
interface Car {  
    void drive();  
}
```

Blueprint  
of an object

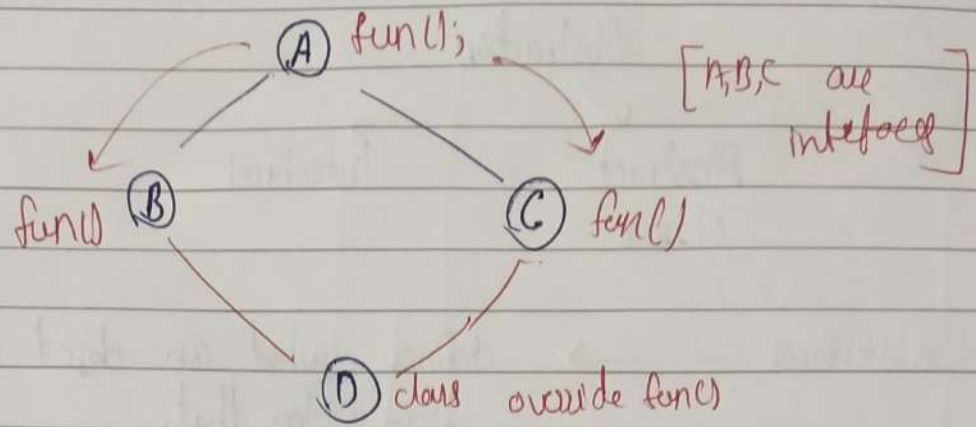
Blueprint  
of  
behaviour

⇒ **Contract**

## \* Diamond Problem



so java didn't implement multiple inheritance through class



~~but in inheritance.~~

but in interfaces multiple inheritance is supported  
as no diamond problem doesn't arrive here

### Java 8 / 9 Interface

- └ default methods
- └ static methods
- └ private methods
- └ static & final variable

### Abstract

- └ concrete method
- └
- └
- └ abstract

\*

### Interface

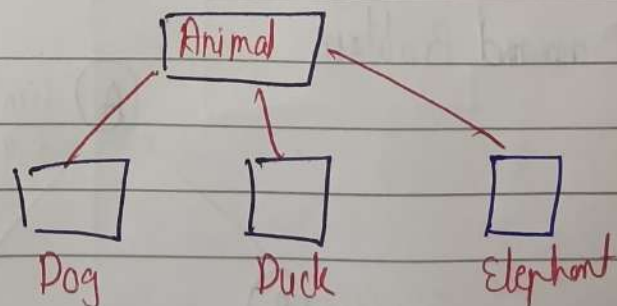
Contract / Rules  
/ Functionalities

eg: Runnable,  
Walkable,  
Payable

(Can-do  
Relationship)

### Abstract class

families of similar  
class



(is-A Relationship)

- ① Fields (X)
- ② Constructors (X)
- ③ Multiple Inheritance (✓)

- Fields (✓)
- Constructors (✓)
- Multiple Inheritance (X)



## Interface

Methods — Public  
                  — private

## Abstract Classes

methods — all access  
                  modifiers  
                  allowed

### ② functional Interfaces

```
interface A {  
    void fun();  
}
```

```
class B implements A {  
    void fun() {  
    }  
}
```

functional programming  
↓  
lambda Expressions  
(we'll study later)

Comparable  
Predicate

### ③ Marker Interfaces

```
interface A { }
```

```
interface Cloneable { }  
interface Serializable { }  
interface RandomAccess { }
```

```
Object — clone();
```

```
Student implements Cloneable  
↳ clone() { }
```

```
interface Animal {  
    void run();  
}
```

class  
treatment  
in compiling

```
javac Animal.java  
↳ Animal.class
```